

LocalWebAuthn — Deployment And Use Options

Perry Kundert

August 9, 2026

Companion to README.org, which builds one application up from nothing. This document is the reference: every option, in the order a deployment meets it. Nothing here is required reading to ship the six routes — but each section is something you will eventually want an answer for.

For the examples, see README-DETAIL-EXAMPLES.org. For machine credentials in full, docs/API-AUTH.org.

1 Who owns what

LocalWebAuthn supplies the lifecycle rules and persistence that ceremony libraries leave to each application, and nothing above them:

Concern	Owner
Application users, roles, authorization	Host
Link approval, delivery, identity proofing	Host
Cookies, exact-origin checks, rate limits	Host
Enrollment grants and atomic replay protection	LocalWebAuthn
Registration and authentication challenges	LocalWebAuthn
Credential metadata, counters, revocation	LocalWebAuthn
Session expiry, touch, logout, revocation	LocalWebAuthn
SQLite, PostgreSQL and D1 schemas	LocalWebAuthn
WebAuthn option generation and verification	SimpleWebAuthn

The split is deliberate: authentication lifecycle is reusable, while identity proofing and business authorization are application policy. docs/RATIONALE.md argues it at length, including the non-goals.

"Self-hosted" describes the relying-party application and its authentication data. It does **not** mean zero dependencies: ceremony verification uses @simplewebauthn/* as library code in your process, HTTPS may be terminated by a proxy or CDN (that is transport, not identity), and a user may

sync passkeys through their OS or password manager — which is not an identity provider for your application, because it never gives LocalWebAuthn a private key.

2 Storage adapters

All three adapters pass one conformance suite, and every `migrate*` call is idempotent, so calling it on each start is fine.

2.1 SQLite (better-sqlite3)

```
import Database from 'better-sqlite3';
import { migrateSqlite, SqliteLocalWebAuthnStore } from '@localwebauthn/server/sqlite';

const database = new Database('application.db');
migrateSqlite(database);
// store: new SqliteLocalWebAuthnStore(database)
```

Real transactions. The adapter enables `PRAGMA foreign_keys = ON` on the connection it is given — in both `migrateSqlite` and the constructor — so keep using that same connection, or the schema's foreign keys stop being enforced.

2.2 PostgreSQL

```
import { Pool } from 'pg';
import { migratePostgres, PostgresLocalWebAuthnStore } from '@localwebauthn/server/postgres';

const pool = new Pool({ connectionString: process.env.DATABASE_URL });
await migratePostgres(pool);
// store: new PostgresLocalWebAuthnStore(pool)
```

Pass a `Pool`, not a single client: transactions need a connection to themselves. The revoke path takes `SELECT ... FOR UPDATE ORDER BY id` on the user's credentials, because under `READ COMMITTED` a conditional `UPDATE` alone let two concurrent revokes each believe another credential remained — which once left an account with zero.

2.3 Cloudflare D1

```
import { D1LocalWebAuthnStore, migrateD1 } from '@localwebauthn/server/d1';
```

```
await migrateD1(env.DB);
// store: new D1LocalWebAuthnStore(env.DB)
```

D1 cannot open a transaction. The adapter uses `batch()` and guards each statement on the preceding row count, which is not the same thing: a mid-batch failure after the credential `INSERT` can leave a credential without its initial session. That row is a normal passkey — the next sign-in creates a session — and no cleanup reconciles it. See `SECURITY.md §D1 Batch Non-Atomicity`.

2.4 Maintenance

```
setInterval(() => void auth.cleanup(), 5 * 60_000);
```

`cleanup()` reaps expired enrollment grants, finished challenges, dead sessions and spent DPoP records, and returns the counts. **It never touches credentials** — those are durable authenticators, removed only by revocation. (Releases 1.0.0 and 1.1.0 did delete idle credentials here; that is the bug that forced 2.0.0.)

3 The six routes

`@localwebauthn/browser` speaks a small JSON protocol over six paths that you implement:

Route (default)	Service call	Cookie
POST <code>/api/auth/enrollment/exchange</code>	<code>exchangeEnrollment(token)</code>	sets enrollment
POST <code>/api/auth/register/options</code>	<code>registrationOptions({...})</code>	sets challenge
POST <code>/api/auth/register/verify</code>	<code>verifyRegistration({...})</code>	clears challenge + enrol
POST <code>/api/auth/login/options</code>	<code>authenticationOptions()</code>	sets challenge
POST <code>/api/auth/login/verify</code>	<code>verifyAuthentication({...})</code>	clears challenge, sets ses
POST <code>/api/auth/logout</code>	<code>revokeSession(token)</code>	clears session

Change the prefix with `new LocalWebAuthnBrowser({ basePath: '/auth' })`, or any single path with the `endpoints` option. Three cookies are involved, all opaque, all hashed before storage, none readable by JavaScript:

- **challenge** — lives only between an `options` call and its `verify`.

- **enrollment** — from link exchange until the first passkey exists.
- **session** — the actual login.

examples/starter-hono/src/auth-routes.ts is a copyable implementation of all six.

4 Cookies, origins and CSRF

Use the helpers rather than re-deriving flags; they are the same rules SECURITY.md requires:

```
import {
  authCookieNames,      // __Host- prefixed on HTTPS, plain on loopback
  cookieAttributes,      // HttpOnly, Path=/, SameSite=Strict, Secure, maxAge from expir
  isExactOrigin,         // scheme+host+port equality, no suffix matching
  isHttpsPublicOrigin,
  parseCookieHeader,     // for plain Node/undici adapters
  serializeCookie,
  serializeClearedCookie,
} from '@localwebauthn/server';
```

authCookieNames and cookieAttributes **throw** for a plain-HTTP publicOrigin that is not loopback. WebAuthn will not run there anyway, so issuing non-Secure cookies would only hide the misconfiguration.

Your route layer still owns CSRF and rate limits. The pattern is one middleware, mounted before your own routes:

```
app.use('/api/*', async (c, next) => {
  c.header('Cache-Control', 'no-store');
  if (c.req.method !== 'GET' && !isExactOrigin(c.req.header('Origin'), publicOrigin)) {
    return c.json({ error: 'invalid_origin' }, 403);
  }
  await next();
});
```

A route that reads no cookie needs no origin check. CSRF requires ambient credentials the browser attaches by itself; a script sends its token in an **Authorization** header and no **Origin** at all, so demanding one would prove nothing — **Origin** is a forbidden request header that only browsers are obliged to set. Machine routes are therefore exempted **by declaration** (the demo calls that list **cookieFreePrefixes**), not by mount order, and the exemption is sound only because those routes read no cookie.

5 Users and suspension

`getUser` is the only place your user table is read. Return `active: false` to suspend someone: every ceremony — issuing and exchanging enrollment links, registration, authentication — and every session resolution refuses them immediately, without revoking anything. Flip it back and their passkeys work again.

When suspending, decide how far to go:

Also call	Effect
nothing	sessions exist but resolve to nothing; fully reversible
<code>revokeUserSessions(userId)</code>	session records ended; passkeys intact
<code>revokeUserAuthentication(userId)</code>	passkeys and pending grants gone; the person must re-enroll

`webAuthnUserHandle` must be a stable, random 32 bytes — `createUserHandle()` produces one. It is what the authenticator stores to name the account, so it must not be an email or an incrementing id, and it must never change for a user who has credentials.

6 Sessions

Defaults, all overridable through `durations`:

Option	Default	Meaning
<code>enrollmentGrantMs</code>	30 minutes	how long an enrollment link may be redeemed
<code>enrollmentSessionMs</code>	10 minutes	from exchange to the passkey existing
<code>challengeMs</code>	5 minutes	between an <code>options</code> call and its <code>verify</code>
<code>sessionIdleMs</code>	30 minutes	inactivity before a session dies
<code>sessionAbsoluteMs</code>	8 hours	total session lifetime regardless of activity

`resolveSession(token)` applies both windows and touches `lastSeenAt`; pass `false` as the second argument to inspect a session without extending it. It returns `null` — never throws — for expired, idle, revoked, revoked-credential and suspended-user cases, so one check covers all five.

A declared credential kind may shorten `sessionAbsoluteMs` for itself (see Credential kinds). The idle window is global on purpose: a per-kind idle window forces the session lookup to filter on the widest window any kind allows and then re-check per kind, which is cost on the hottest path in the package for something a host can get by choosing a smaller absolute lifetime.

7 Revocation

Call	Ends
<code>revokeSession(token)</code>	one session
<code>revokeUserSessions(userId, { exceptSessionToken })</code>	every session, optionally
<code>revokeUserSessions(userId, { kinds })</code>	only sessions opened by
<code>revokeCredential(userId, credentialId, { allowLastCredential })</code>	one credential and its se
<code>revokeCredentialTree(userId, credentialId, { kinds })</code>	a credential and everyth
<code>revokeUserAuthentication(userId, { kinds })</code>	credentials, sessions, and

Two behaviours to know:

The last-credential guard is scoped by kind. `revokeCredential` refuses to remove a person's only credential **of that kind** unless `allowLastCredential`: **true**. Without the scoping, someone holding one passkey and one API credential could have their only passkey revoked and be told it worked.

`revokeCredentialTree` is the remediation primitive. A stolen session can enroll a passkey of its own — that is the intended "add a passkey" feature for a person, so it cannot simply be forbidden — and afterwards the attacker's credential is indistinguishable from a legitimate one. Revoking the subtree is the only remedy. It revokes with `allowLastCredential` and may empty the account: stopping short would leave a half-revoked tree, which after a compromise is worse than re-enrolling.

Unscoped, it crosses kinds, and that is usually a surprise worth knowing: every API credential a person provisions has **their passkey** as its parent, so revoking a suspected passkey's tree stops their scripts too. After a compromise that is right. When it is not what you meant, pass kinds — which restricts what is revoked, not what is walked, so sparing a credential does not silently spare what that credential enrolled.

8 Designing recovery

A user will lose their phone. What you do next is the most important security decision in a passkey deployment, and the one this package refuses to make for you.

8.1 Why the bar has to be high

Account security is the weaker of the passkey and the recovery path. A passkey is origin-bound, unphishable and useless to whoever steals

your database. An attacker looking at that will not attack it — they will contact your support address and say they got a new phone. Every hour of phishing resistance is refunded the moment recovery is easier than the front door. Well-defended accounts are overwhelmingly lost to help-desk social engineering and SIM swaps, not to broken cryptography.

Recovery must be at least as strong as enrollment. If a new employee gets their link only after a manager confirms them in person, then recovery that needs a matching email address is not recovery — it is a bypass of onboarding, available to anyone who can read that mailbox.

Recovery is rare, so it may be slow. Sign-in happens constantly and must be frictionless. Recovery happens perhaps once per user every few years. That asymmetry is a gift: you can afford a human in the loop, a callback, a 24-hour delay, a video call. Almost nowhere else can you buy that much security for that little aggregate friction. Do not reflexively automate it.

The enrollment link is a bearer capability. Whoever opens it first gets a working passkey. Identity-proofing the requester is half the job; the delivery channel is the other half. A perfect identity check followed by an email to an attacker’s inbox is an account takeover.

8.2 Pattern: administrator-initiated

Best whenever a human can vouch for the person. The strongest version ships **no self-service recovery endpoint at all**.

1. The user contacts an administrator out of band.
2. The administrator confirms them through a channel that proves **personhood**, not account control: a video call where they recognise the face, an in-person visit, a manager vouching. A caller who knows the employee’s manager, start date and desk number has proven only that they did some research.
3. Revoke, then issue, then deliver over a channel bound to the person rather than the account — an existing session on a managed device, or hand to hand.

```
// Order matters. revokeUserAuthentication() revokes pending enrollment grants,
// so issuing first would destroy the link you just made.
await auth.revokeUserAuthentication(user.id);
const { enrollmentUrl } = await auth.issueEnrollment(user.id, {
```

```
    approvedByUserId: administrator.id,  
  });
```

`approvedByUserId` is stored on the grant and copied onto any credential it creates, so every recovery has a named person accountable for it. That record is worth more than it looks: a recovery nobody signed off on is exactly the shape a social-engineering attack leaves behind.

8.3 Pattern: multiple independent channels

For consumer systems with no administrator, require proof of control over two channels the user registered **before** the loss — typically a verified email address and phone number. Verify both before issuing anything, and send the link only to the address already on file.

Three failure modes to design against:

- **The channels must be genuinely independent.** If the phone number is the reset method for the email account, they are one factor wearing two hats.
- **A phone number is weaker than it feels.** Numbers get ported and SIM swaps are cheap and targeted. Treat "controls the number today" as one piece of evidence — awkward precisely in the case you are designing for, where the legitimate user genuinely does have a new phone and possibly a new SIM.
- **Never let the request choose the destination.** Address and number come from the user record, not the form.

8.4 Whichever pattern you choose

- **Revoke before you issue.** If the device was stolen rather than lost, this is what evicts whoever has it.
- **Shorten the window.** Drop `durations.enrollmentGrantMs` well below 30 minutes for recovery links; they should be used within minutes of a live conversation.
- **Tell the user on every channel you have,** at initiation and completion. A legitimate user who did not ask for this is your best intrusion detector, and a cool-off period gives them time to object.

- **Keep the audit trail.** `onEvent` emits `enrollment.issued`, `enrollment.revoked` and `enrollment.completed` with a `grantId` you can join against the grant's `approved_by_user_id`. Persist them; recovery is the flow you will most want to reconstruct later.
- **Rate limit it.** Recovery is low-volume by nature, so a tight limit costs real users nothing and makes attempts expensive and visible.

8.5 The cheapest recovery is the one you never run

If the user still has one working passkey, this is not recovery at all:

```
await auth.registerPasskey('Backup security key');
```

Prompt for a second passkey during initial enrollment — a hardware key in a drawer, or a second device — and most of the above stays unused.

9 Self-serve signup

Nothing requires a human in the loop. An automated signup that first proves something about the person can issue the grant itself; the constraint is that email or SMS must not remain a standing route into the account afterwards, or the mailbox is back in front of the passkey.

The package ships the state helpers, not the policy:

```
import {
  describeSignupPhase,
  nextSignupStep,
  SELF_SERVE_SIGNUP_STEPS,
  signupPhase,
} from '@localwebauthn/server';

const phase = signupPhase({
  hasActiveCredential,
  hasPendingEnrollmentGrant,
  hasEnrollmentSession,
});
// 'created' | 'enrollment_issued' | 'enrollment_exchanged' | 'enrolled'
const step = nextSignupStep(phase); // { action, reason }
```

`SELF_SERVE_SIGNUP_STEPS` is the checklist in order, including the two easy ones to get wrong: verify two independent channels **before** creating durable access, and deliver to a bound address rather than one supplied in the request.

The worked implementation — capability-free proof links, claim-on-reopen so the person finishes on whichever device they prefer, and for accounts that already have passkeys a waiting period with "this wasn't me" vetoes from any channel plus cancellation on any successful passkey sign-in — is `examples/channels/src/signup.ts`. The demo exercises all of it with simulated delivery; the channels examples supply real senders.

10 Credential kinds

A credential carries a host-defined **kind**. Undeclared kinds — including `null`, which every credential created before this feature has — behave exactly as they did before, so adding the option changes nothing until you opt in.

```
credentialKinds: {
  service: {
    interactive: false,           // may not sign in where no kind is named
    canRegister: false,         // may not enroll another credential
    sessionAbsoluteMs: 15 * 60_000, // shorter total session lifetime
  },
}
```

The reason **kind** exists is that **only server-chosen facts are controls**. A software client holds its own private key, so it can assert any **origin**, set the user-verified and backup flags, and report any signature counter. **kind**, the challenge, revocation state and heritage are server rows; everything else in an assertion is a claim the signer makes about itself.

Two limits to internalise:

- **canRegister: false** closes the **session** registration path. It cannot close the **grant** path, which is authorized purely by possession of a single-use token with no session to inspect. Confinement there comes from the grant's own kind (`issueEnrollment(userId, { credentialKind: 'service' })`) plus your middleware refusing non-interactive kinds — `auth.interactiveKind(kind)`.
- Kind filters on the bulk revokes let you sign a person out of their devices without stopping their nightly export. The scoped `revokeUserAuthentication`

also revokes pending grants **of those kinds**, because a live grant of a revoked kind is standing authorization to re-enroll straight back in.

11 Machine credentials and DPoP

The full design is docs/API-AUTH.org; README.org §"A Passkey for a script" is the working code. What belongs here is the operational surface.

`verifyDpop` verifies an RFC 9449 proof against the session's credential. There is no per-session key material: the expected thumbprint is **derived** from `credentials.public_key`, so the per-request check reduces to "signed by the same key that opened this session". The only new state is a `jti` replay cache.

Server-issued nonces are opt-in, because they cost the client something real — it must retain the latest DPoP-Nonce and retry once when challenged:

```
dpopNonce: { rotationMs: 60_000 }
```

They are stored in the database, keyed by time slot, so every server in a deployment converges on one value per slot without a shared secret — whichever inserts first decides, the rest read it back. A nonce is **not** a secret; the server hands the current one to any caller. Its only property is being unguessable **in advance**, which is what stops a key holder pre-generating proofs. Both the current and previous slot are accepted so a rotation landing mid-flight does not reject a proof built moments earlier.

`dpopChallenge(nonce)` returns the `WWW-Authenticate / DPoP-Nonce` header pair for the `dpop_nonce_required` case, so hosts stop hand-rolling four steps.

A custom store needs the three DPoP methods only if you use this feature: they live in a separate `LocalWebAuthnDpopStore` contract, and the service checks for them where it uses them, throwing `invalid_configuration` naming the missing ones. Every official adapter implements both.

12 Credential heritage

Every credential this package creates was authorized either by an enrollment grant or by an existing credential's session. Four columns record which:

Column	Meaning
<code>createdVia</code>	' enrollment ', ' session ', or null for pre-2.x rows
<code>parentCredentialId</code>	the credential whose session authorized this one
<code>grantId</code>	the enrollment grant it was redeemed from
<code>approvedByUserId</code>	who approved that grant

```
await auth.credentialLineage(userId, credentialId);    // root first, back to the enro
await auth.credentialDescendants(userId, credentialId); // index 0 is the credential i
```

`grantId` and `approvedByUserId` are deliberately **not** foreign keys: grants are reaped by `cleanup`, so these are copied facts about the past. `parentCredentialId` may be a real foreign key only because credentials are never deleted.

13 Audit events

`onEvent` receives every lifecycle event. It is best-effort — persist what matters rather than treating the callback as the record:

```
onEvent: (event) => void auditLog.write(event),
```

Event	Emitted when
<code>enrollment.issued</code>	a grant is created
<code>enrollment.exchanged</code>	a token is redeemed for an enrollment session
<code>enrollment.completed</code>	a passkey is registered against that grant
<code>enrollment.revoked</code>	a pending grant is superseded or revoked
<code>credential.registered</code>	a passkey is created
<code>credential.authenticated</code>	an assertion verifies
<code>credential.revoked</code>	a credential is revoked
<code>session.created</code>	a session begins
<code>session.revoked</code>	a session ends
<code>user.sessions_revoked</code>	bulk session revoke, with <code>count</code> and any <code>kinds</code>
<code>user.authentication_revoked</code>	bulk credential revoke, with any <code>kinds</code>

Return values are the durable record where one exists: `issueEnrollment` returns `supersededGrantIds` precisely so a host can write the replacement down without depending on the event.

14 Errors

Every failure is a `LocalWebAuthnError` with a `code` and an HTTP `status`. Map them once:

Code	Status	Means
<code>invalid_configuration</code>	500	the host asked for something inconsistent
<code>invalid_enrollment</code>	400/403/404	bad, expired, consumed or revoked link; unknown user
<code>enrollment_not_authorized</code>	403	no valid grant or session authorizes this registration
<code>invalid_ceremony</code>	400	challenge missing, expired or already used
<code>registration_failed</code>	400	the attestation did not verify
<code>authentication_failed</code>	401	the assertion did not verify
<code>registration_not_permitted</code>	403	this kind is <code>canRegister: false</code>
<code>credential_not_found</code>	404	no such credential for this user
<code>last_credential</code>	409	would remove the person's only credential of that kind
<code>invalid_dpop_proof</code>	401	absent, malformed, replayed, or wrong key
<code>dpop_nonce_required</code>	401	answer with <code>dpopChallenge()</code> and let the client retry

Do not surface the `reason` detail attached to DPoP messages: it distinguishes "wrong key" from "replayed".

15 Migrations and versioning

The service API, store contract and database schema follow SemVer; breaking changes arrive only in a new major version with notes in `docs/MIGRATING.md`.

Schema upgrades are versioned and applied in place: `migrateSqlite`, `migratePostgres` and `migrateD1` read the stored version and run only what is missing. A database at version 1 — the only version ever published — upgrades on the next start.

Two traps if you extend the schema yourself: `LOCALWEBAUTHN_SCHEMA_SQL` must contain no `--` comments (the statement splitter collapses whitespace and would comment out the following statement), and a new **table** needs only its name in that version's `newTables`, because the upgrade path lifts its `CREATE TABLE IF NOT EXISTS` and indexes from the full schema in the right dialect.

16 Reading the code

Every SQL statement the package executes is collected in `packages/server/src/queries.ts`, so a reviewer can read the data layer in one file. `SECURITY.md` states the

security boundary, the host's duties, what is stored and what is not, and how to report a vulnerability.

If you are auditing and do not issue API credentials, `packages/server/src/dpop.ts` and the `credentialKinds` handling are the parts you can skip.